

Este tutorial fornece um guia passo a passo para a implementação de autenticação JWT (JSON Web Token) em uma aplicação NestJS, utilizando Prisma e PostgreSQL, baseando-se nos códigos do repositório `backend-nestjs`. O módulo de autenticação é gerado com o comando `nest g resource`.

Tutorial de Autenticação JWT com NestJS

Este guia é a continuação do desenvolvimento do seu CRUD de usuários. Aqui, transformaremos sua aplicação em um sistema seguro onde apenas usuários autenticados podem acessar determinados recursos.

Etapa 1: Instalação de Dependências de Segurança

Descrição: Para implementar a autenticação, precisamos de bibliotecas que lidem com a criação de tokens e a criptografia de senhas.

- **Conceitos:**
 - **Passport:** Middleware de autenticação padrão para Node.js.
 - **JWT:** Um padrão para transmitir informações de forma segura entre as partes como um objeto JSON.
 - **Bcrypt:** Biblioteca para criar hashes de senhas, garantindo que elas não sejam salvas em texto puro no banco de dados.

Comando Bash:

Bash

```
npm install @nestjs/passport passport @nestjs/jwt passport-jwt bcrypt
npm install -D @types/passport-jwt @types/bcrypt
```

Etapa 2: Criptografia de Senha no Registro

Descrição: Ao criar um usuário no `UserService`, a senha deve ser transformada em um "hash" seguro antes de ser salva no banco. Também adicionamos o método `findByEmail`, que será usado no login.

Código Fonte (`src/users/users.service.ts`):

TypeScript

```
import { Injectable } from '@nestjs/common';
import { PrismaService } from '../prisma/prisma.service';
import { CreateUserDto } from '../dto/create-user.dto';
import * as bcrypt from 'bcrypt'; // Biblioteca para hash de senha

@Injectable()
export class UserService {
  constructor(private prisma: PrismaService) {}

  async create(createUserDto: CreateUserDto) {
    // Gera um salt e cria o hash da senha enviada pelo DTO
    const salt = await bcrypt.genSalt();
    const hash = await bcrypt.hash(createUserDto.password, salt);

    // Salva o usuário no banco com a senha criptografada
    return this.prisma.user.create({
      data: { ...createUserDto, password: hash },
    });
  }

  // Método essencial para buscar usuário pelo e-mail durante o login
  async findByEmail(email: string) {
    return this.prisma.user.findUnique({
      where: { email },
    });
  }
}
```

```
});  
}  
// ... demais métodos do CRUD  
}
```

Etapa 3: Gerando o Recurso de Autenticação com `nest g resource`

Descrição: Em vez de criar os arquivos manualmente, usamos o gerador do NestJS para criar a estrutura base do módulo `auth` (module, controller e service) e já registrá-lo automaticamente no `AppModule`.

Comando Bash:

Bash

```
nest g resource auth
```

O CLI fará duas perguntas:

1. **What transport layer do you use?** → selecione **REST API**.
2. **Would you like to generate CRUD entry points?** → responda **No**.

Por que "No" nos entry points do CRUD? A autenticação não é um CRUD: não existe "criar, listar, atualizar e remover autenticações". Respondendo **No**, o CLI gera apenas o essencial, sem DTOs e entidades desnecessários.

Estrutura gerada:

```
src/auth/  
├─ auth.controller.ts  
├─ auth.controller.spec.ts  
├─ auth.module.ts  
├─ auth.service.ts  
└─ auth.service.spec.ts
```

Os arquivos gerados vêm praticamente vazios. Nas próximas etapas vamos preenchê-los e criar dois arquivos adicionais: `dto/login.dto.ts` e `jwt.strategy.ts`.

Dica: se não quiser os arquivos de teste (`.spec.ts`), use `nest g resource auth --no-spec`.

Etapa 4: Criando o DTO de Login

Descrição: O DTO define o formato e as regras de validação dos dados enviados na rota de login (e-mail e senha).

Crie a pasta e o arquivo manualmente:

Bash

```
mkdir src/auth/dto
```

Código Fonte (`src/auth/dto/login.dto.ts`):

TypeScript

```
import { ApiProperty } from '@nestjs/swagger';  
import { IsEmail, IsNotEmpty, IsString } from 'class-validator';  
  
export class LoginDto {  
  @ApiProperty({ example: 'joao@email.com', description: 'E-mail do usuário' })  
  @IsEmail()  
  email!: string;  
  
  @ApiProperty({ example: 'senha123', description: 'Senha do usuário' })  
  @IsString()  
  @IsNotEmpty()  
  password!: string;  
}
```

```
password!: string;
}
```

Etapa 5: Implementando o AuthService

Descrição: Abra o `AuthService` gerado e implemente a lógica que valida as credenciais do usuário e gera o token de acesso.

- **Conceitos:**
 - **Payload:** O conteúdo "útil" dentro do token (ex: ID e e-mail do usuário).
 - **Secret Key:** Uma chave secreta usada para assinar o token e garantir sua integridade.

Código Fonte (`src/auth/auth.service.ts`):

TypeScript

```
import { Injectable, UnauthorizedException } from '@nestjs/common';
import { UsersService } from '../users/users.service';
import { JwtService } from '@nestjs/jwt';
import * as bcrypt from 'bcrypt';
import { LoginDto } from '../dto/login.dto';

export interface JwtPayload { sub: number; email: string; }

@Injectable()
export class AuthService {
  constructor(
    private usersService: UsersService,
    private jwtService: JwtService,
  ) {}

  async login(loginDto: LoginDto) {
    // Busca o usuário pelo e-mail
    const user = await this.usersService.findByEmail(loginDto.email);

    // Compara a senha digitada com o hash salvo no banco
    if (!user || !(await bcrypt.compare(loginDto.password, user.password))) {
      throw new UnauthorizedException('E-mail ou senha incorretos');
    }

    // Define o conteúdo do token
    const payload: JwtPayload = { sub: user.id, email: user.email };

    return {
      access_token: this.jwtService.sign(payload), // Gera o JWT assinado
    };
  }
}
```

Etapa 6: Implementação da Estratégia JWT

Descrição: Define como a aplicação deve extrair e validar o token enviado nas requisições subsequentes. Este arquivo não é gerado pelo `nest g resource`, então crie-o manualmente dentro de `src/auth/`.

- **Conceito: Bearer Token:** O padrão onde o token é enviado no cabeçalho `Authorization` da requisição HTTP como `Bearer <token>`.

Código Fonte (`src/auth/jwt.strategy.ts`):

TypeScript

```
import 'dotenv/config';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { PassportStrategy } from '@nestjs/passport';
import { Injectable } from '@nestjs/common';
```

```

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    const secret = process.env.JWT_SECRET;
    if (!secret) throw new Error('JWT_SECRET não definida');

    super({
      // Extrai o token do cabeçalho de autorização como Bearer Token
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: secret,
    });
  }

  // Se o token for válido, o NestJS anexa este retorno ao objeto da requisição (req.user)
  validate({ sub, email }: { sub: number; email: string }) {
    return { userId: sub, email };
  }
}

```

Adicione a chave secreta no arquivo `.env`:

```
JWT_SECRET="troque-por-uma-chave-longa-e-aleatoria"
```

Importante: nunca use uma chave fixa no código como valor padrão. Se a variável não estiver definida, a aplicação deve falhar ao iniciar.

Etapa 7: Criando a Rota de Login (AuthController)

Descrição: Abra o `AuthController` gerado e crie a rota `POST /auth/login`, que recebe o `LoginDto` e devolve o token.

Código Fonte (`src/auth/auth.controller.ts`):

TypeScript

```

import { Body, Controller, HttpStatusCode, HttpStatus, Post } from '@nestjs/common';
import { ApiOperation, ApiResponse, ApiTags } from '@nestjs/swagger';
import { AuthService } from '../auth.service';
import { LoginDto } from '../dto/login.dto';

@ApiTags('auth')
@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  @Post('login')
  @HttpCode(HttpStatus.OK) // POST retorna 201 por padrão; no login o correto é 200
  @ApiOperation({ summary: 'Autenticar usuário e gerar token JWT' })
  @ApiResponse({ status: 200, description: 'Login realizado com sucesso.' })
  @ApiResponse({ status: 401, description: 'E-mail ou senha incorretos.' })
  login(@Body() loginDto: LoginDto) {
    return this.authService.login(loginDto);
  }
}

```

Etapa 8: Configurando os Módulos

Descrição: Agora conectamos as peças. O `AuthModule` precisa importar o `UsersModule` (para usar o `UsersService`), o `PassportModule` e o `JwtModule`, além de registrar a `JwtStrategy` como provider.

Código Fonte (`src/auth/auth.module.ts`):

TypeScript

```
import 'dotenv/config';
import { Module } from '@nestjs/common';
import { JwtModule } from '@nestjs/jwt';
import { PassportModule } from '@nestjs/passport';
import { UsersModule } from '../users/users.module';
import { AuthController } from './auth.controller';
import { AuthService } from './auth.service';
import { JwtStrategy } from './jwt.strategy';

@Module({
  imports: [
    UsersModule, // Para usar o UsersService no AuthService
    PassportModule,
    JwtModule.register({
      secret: process.env.JWT_SECRET,
      signOptions: { expiresIn: '1h' }, // Tempo de validade do token
    }),
  ],
  controllers: [AuthController],
  providers: [AuthService, JwtStrategy],
})
export class AuthModule {}
```

O UsersModule precisa **exportar** o UsersService para que outros módulos possam utilizá-lo.

Código Fonte (src/users/users.module.ts):

TypeScript

```
import { Module } from '@nestjs/common';
import { PrismaModule } from '../prisma/prisma.module';
import { UsersService } from './users.service';
import { UsersController } from './users.controller';

@Module({
  imports: [PrismaModule],
  controllers: [UsersController],
  providers: [UsersService],
  exports: [UsersService], // Expõe o UsersService para o AuthModule
})
export class UsersModule {}
```

O nest g resource já adiciona o AuthModule no array imports do src/app.module.ts. Confirme que ele está lá; se não estiver, adicione manualmente.

Etapa 9: Proteção das Rotas (Controller)

Descrição: Agora aplicamos o AuthGuard para garantir que apenas usuários logados acessem as rotas de usuários.

- **Conceito: Guard:** Classe que determina se uma requisição pode prosseguir ou não com base em condições (como estar autenticado).

Atenção: se aplicarmos o @UseGuards na classe inteira, a rota POST /users também ficará protegida e ninguém conseguirá se cadastrar (para criar o primeiro usuário seria necessário já ter um token). Por isso, aplicamos o guard **por rota**, deixando o cadastro público.

Código Fonte (src/users/users.controller.ts):

TypeScript

```
import { Controller, Get, Post, Body, Patch, Param, Delete, UseGuards } from '@nestjs/common';
import { ApiBearerAuth, ApiOperation, ApiResponse, ApiTags } from '@nestjs/swagger';
import { AuthGuard } from '@nestjs/passport';
import { UsersService } from './users.service';
import { CreateUserDto } from './dto/create-user.dto';
import { UpdateUserDto } from './dto/update-user.dto';
```

```

@ApiTags('users')
@Controller('users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  // Rota pública: permite o cadastro de novos usuários
  @Post()
  @ApiOperation({ summary: 'Criar um novo usuário' })
  @ApiResponse({ status: 201, description: 'Usuário criado com sucesso.' })
  @ApiResponse({ status: 400, description: 'Dados inválidos.' })
  create(@Body() createUserDto: CreateUserDto) {
    return this.usersService.create(createUserDto);
  }

  // Rotas protegidas: exigem o token JWT
  @ApiBearerAuth('token') // Configura o Swagger para enviar o Token JWT
  @UseGuards(AuthGuard('jwt'))
  @Get()
  @ApiOperation({ summary: 'Listar todos os usuários' })
  findAll() {
    return this.usersService.findAll();
  }

  @ApiBearerAuth('token')
  @UseGuards(AuthGuard('jwt'))
  @Get(':id')
  @ApiOperation({ summary: 'Buscar um usuário pelo ID' })
  findOne(@Param('id') id: string) {
    return this.usersService.findOne(+id);
  }

  @ApiBearerAuth('token')
  @UseGuards(AuthGuard('jwt'))
  @Patch(':id')
  @ApiOperation({ summary: 'Atualizar um usuário' })
  update(@Param('id') id: string, @Body() updateUserDto: UpdateUserDto) {
    return this.usersService.update(+id, updateUserDto);
  }

  @ApiBearerAuth('token')
  @UseGuards(AuthGuard('jwt'))
  @Delete(':id')
  @ApiOperation({ summary: 'Remover um usuário' })
  remove(@Param('id') id: string) {
    return this.usersService.remove(+id);
  }
}

```

Etapa 10: Configuração do Swagger com Autenticação

Descrição: Ajustamos o arquivo principal para que a interface do Swagger permita inserir o token e testar as rotas protegidas. Mantemos também o `ValidationPipe` global, necessário para validar o `LoginDto` e o `CreateUserDto`.

Código Fonte (`src/main.ts`):

TypeScript

```

import { ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.useGlobalPipes(new ValidationPipe()); // Ativa validação dos DTOS

```

```

const config = new DocumentBuilder()
  .setTitle('User CRUD API')
  .setDescription('Documentação da API com NestJS, Prisma e JWT')
  .setVersion('1.0')
  .addTag('users')
  .addTag('auth')
  .addBearerAuth( // Adiciona o campo de autenticação no Swagger
    {
      type: 'http',
      scheme: 'bearer',
      bearerFormat: 'JWT',
      name: 'JWT',
      in: 'header',
    },
    'token',
  )
  .build();

const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('api', app, document);

await app.listen(3000);
}
bootstrap();

```

Estrutura Final do Módulo de Autenticação

```

src/
├── auth/
│   ├── dto/
│   │   └── login.dto.ts          (criado manualmente)
│   ├── auth.controller.ts       (gerado por nest g resource)
│   ├── auth.module.ts           (gerado por nest g resource)
│   ├── auth.service.ts          (gerado por nest g resource)
│   └── jwt.strategy.ts           (criado manualmente)
└── users/
    ├── users.controller.ts       (rotas protegidas com AuthGuard)
    ├── users.module.ts           (exporta UsersService)
    └── users.service.ts          (hash de senha + findByEmail)

```

Como Testar:

1. **Iniciar:** Execute `npm run start:dev` e acesse `http://localhost:3000/api`.
2. **Registrar:** Crie um usuário usando a rota `POST /users` (rota pública; a senha será criptografada automaticamente).
3. **Login:** Use a rota `POST /auth/login` com o e-mail e a senha cadastrados. Copie o valor de `access_token` retornado.
4. **Autorizar:** No Swagger, clique no botão verde "Authorize" no topo, cole o token e confirme.
5. **Acessar:** Tente listar os usuários em `GET /users`. Agora você terá permissão! Sem o token, a rota retorna `401 Unauthorized`.